

From one-off notebooks to repeatable, challengeable experiments

A2-NLP Project Team · CSED 504 · University of Washington · Summer 2026



TAKEAWAY The factory made hundreds of model comparisons affordable and auditable—then gave collaborators enough control to disprove results that initially looked convincing.

SYSTEM GOAL

Why a factory?

A scaling study varies data, compute, model size, language, seed and learning rate. Managing that grid by notebook state and filenames is not reproducibility. The factory turns each cell into a self-describing job, checkpoint and result record.

Prepare once → inspect and fingerprint → estimate on the actual GPU → pretrain one cell or queue a fleet → read curves and results through the same API. Interactive exploration stays in notebooks; expensive work moves to restartable processes.

MEASURED SPEEDUP

2.07× — and only 1.32× of it is efficiency

Both changes were available on day one and neither is clever. The batch was simply too small, and the second card had never been used. Utilization went from one card busy and one idle to 91% and 93%.

Efficiency and parallelism are reported separately.

NOTEBOOK / QUEUE BOUNDARY

Preparation is under a minute. Training is an hour and a half.

Which is the whole reason the factory is shaped the way it is: everything cheap happens interactively in a notebook, and everything expensive goes into a queue that runs while nobody is watching.

Cheap stages stay interactive; training becomes a job.

PUBLIC SURFACE

A tool nobody can find does not exist

```
THE WHOLE INTERFACE
import mlm_api as factory

bits_per_char(...)
corpus_info(name)
stream(name, ...)
estimate(name, cells, ...)
pretrain(name, tokens, steps, ...)
random_init(name, ...)
random_init_like(model_id, ...)
results(...)
curve(tag)
```

9 functions to import nothing else in the folder

361 you import — mlm_api.py

1,634 of factory behind it

14,537 in the folder, 53 files

Leon cloned the repository, read the documentation, and asked whether there was an interface he was supposed to be using. There was. The folder's front page was titled with a different study and mlm_api first appeared on line 25, one row of a second table. Every word of it accurate. Still unfindable — and that is a failure of the interface, not of the reader.

Nine stable calls hide the machinery.

OPERATIONS

training nothing training right now

swap62k 3 of 3 done · 0 running · 0 not started

swap62k · Yoruba · 69.1M tokens of test · 33.8M model · 62.5k steps	1024M updates	DONE
swap62k · Yoruba · 69.1M tokens of test · 33.8M model · 62.5k steps · seed 1	1024M updates	DONE
swap62k · Yoruba · 69.1M tokens of test · 33.8M model · 62.5k steps · seed 2	1024M updates	DONE

Finished. Estimated from this machine's own finished runs — 86M 100k tok/s · 33.8M 349k tok/s, scheduled across 2 cards.

Reading this: the blue line is loss on held-out test (lower is better), orange is training loss. At the same data and the same compute, how much does the language matter? Best so far: French at 2.54, 1.92 rats of loss ahead of Mandarin.

Where each one lands (lower is better)

How each one gets there (dot = stopped improving)

Queue, GPU state and comparable run records.

What made it trustworthy

Every setting that can move a number belongs in the record. Corpora and vocabularies carry fingerprints; run reuse refuses mismatched settings. The current evidence base contains 197 pretraining runs, 278 fine-tuning records and 892 individual fine-tuning runs.

At three seeds per arm, a mean difference must be about 2.27× the pooled sample spread to clear a two-sided 0.05 t-threshold. An exact permutation test cannot return below p=0.10 with 3 vs 3. A pre-registered sample size limits what a run grid is allowed to claim.

The claims gate currently reports 6 supported, 2 unsupported and 1 underpowered claim. That is a feature: numbers regenerate from records, comparative claims are tested against their nulls, and failed claims remain visible instead of being silently rewritten.